Partie 6 : Base de Données & Eloquent ORM

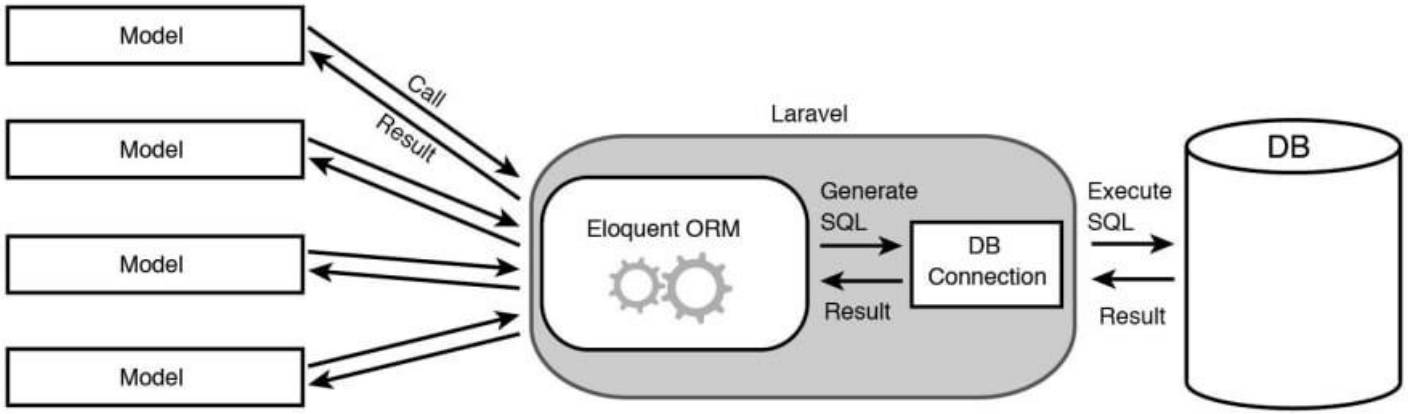
6. Base de Données & Eloquent ORM

Dans une application web dynamique, les données doivent être stockées, organisées et manipulées. Laravel facilite la gestion des bases de données grâce à :

- ✓ Un système de configuration simple
- ✓ Les migrations pour créer les tables
- ✓ Les seeders pour insérer des données
- ✓ Eloquent ORM pour interagir avec la base de données en utilisant des objets PHP

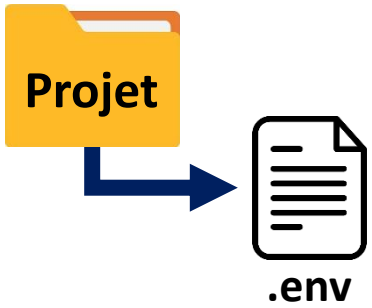
Laravel supporte plusieurs SGBD:

- ✓ MySQL
- ✓ PostgreSQL
- ✓ SQLite
- ✓ SQL Server



Configuration de la BD

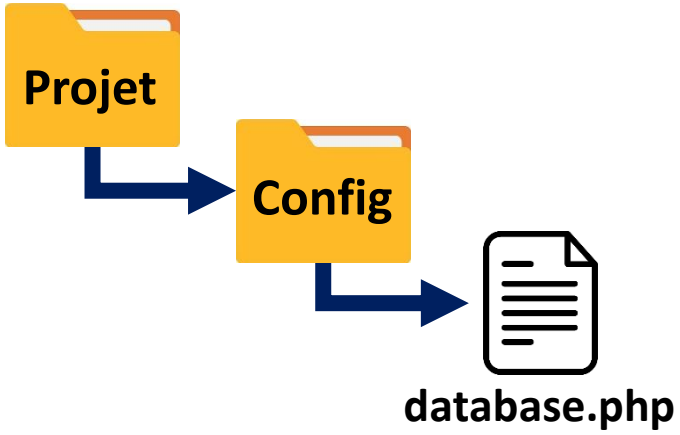
La configuration se fait dans le fichier ***.env***

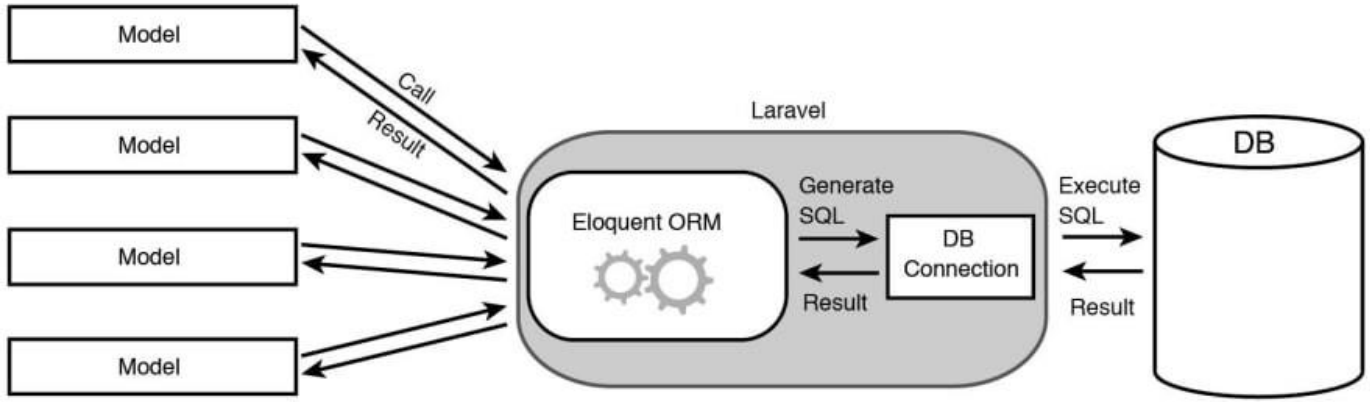



```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel_db
DB_USERNAME=root
DB_PASSWORD=
```

Base de données	Driver Laravel	Port par défault
MySQL	mysql	3306
MariaDB	mysql	3306
PostgreSQL	pgsql	5432
SQLite	sqlite	------
SQL Server	sqlsrv	1433

Puis Laravel utilise ces informations dans :





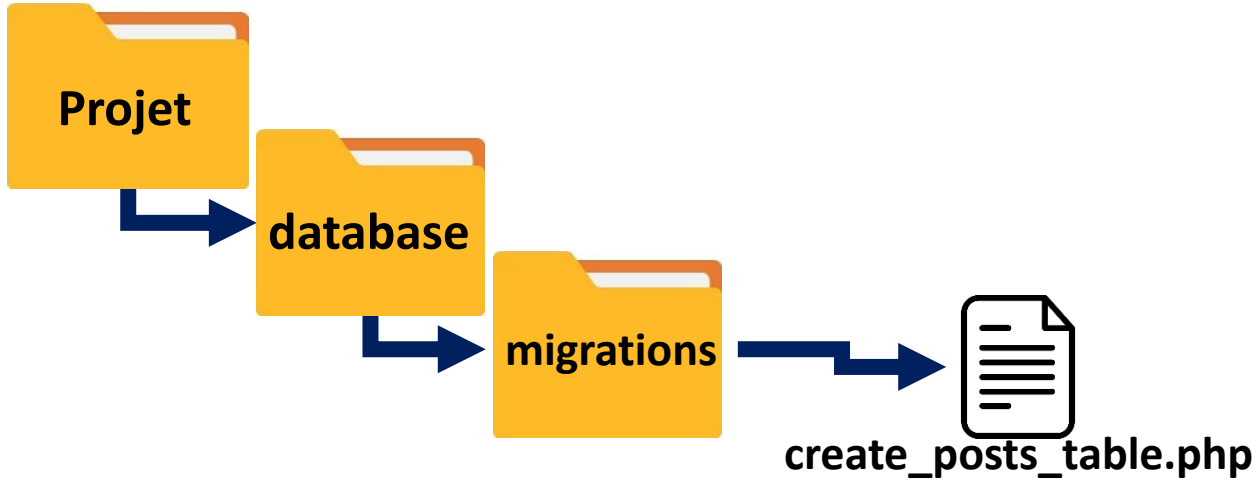
Migrations

Les migrations permettent de créer et modifier les tables de la base de données avec du code PHP.

📌 Avantages :

- ✅ Versionnement de la base de données
- ✅ Travail collaboratif
- ✅ Reproduction facile du projet

php artisan make:migration create_posts_table




```php
public function up()
{
    Schema::create('posts', function (Blueprint $table) {
        $table->id();
        $table->string('title');
        $table->text('content');
        $table->timestamps();
    });
}
```

Exécuter les migrations:
php artisan migrate

Supprimer les tables :
php artisan migrate:rollback

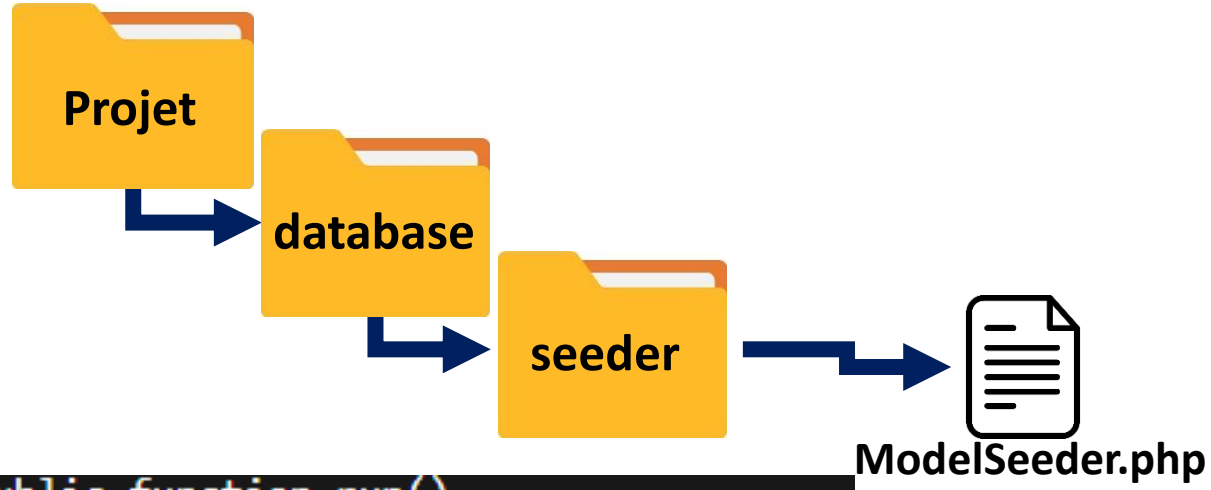
Recréer les tables :
php artisan migrate:fresh

6. Base de Données & Eloquent ORM

Seeders et Factories

Les Seeders permettent d'insérer des données automatiquement dans la base.

php artisan make:seeder ModelSeeder



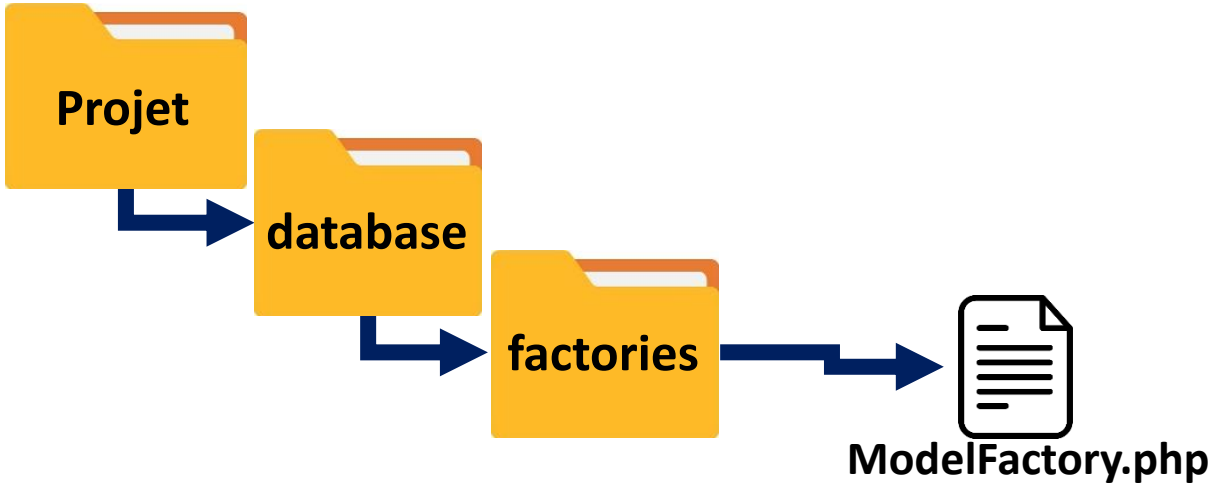

```php
public function run()
{
    DB::table('posts')->insert([
        'title' => 'Premier Post',
        'content' => 'Contenu du post'
    ]);
}
```

User::factory()->count(5)->create();

Exécuter :
php artisan db:seed

Les Factories permettent de générer beaucoup de données de test.

php artisan make:factory ModelFactory



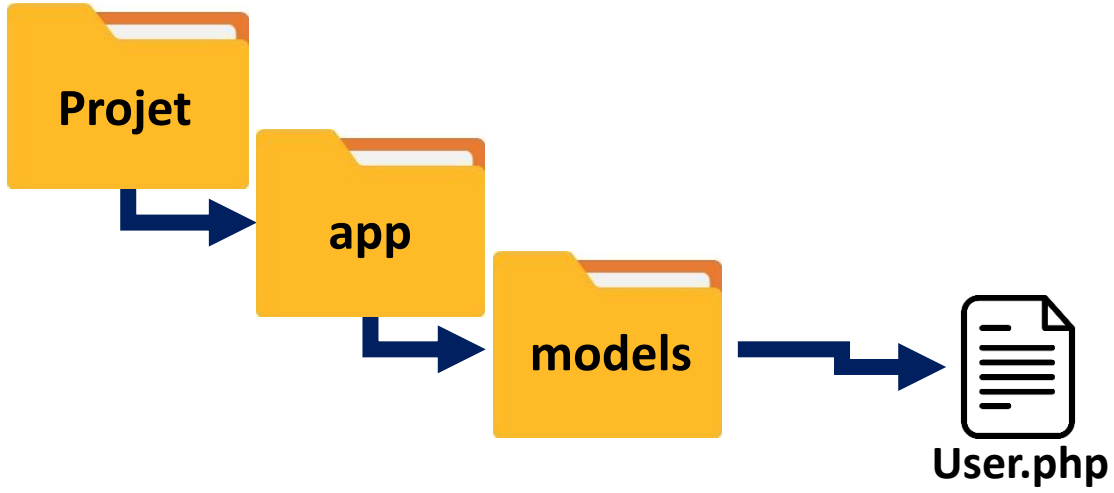

```php
public function definition(): array
{
    return [
        'name' => fake()->name(),
        'email' => fake()->unique()->safeEmail(),
        'email_verified_at' => now(),
        'password' => static::$password ??= Hash::make('password'),
        'remember_token' => Str::random(10),
    ];
}
```

Méthode	Description	Exemple
fake()->sentence()	Génère une phrase	"Laravel is a powerful framework."
fake()->paragraph()	Génère un paragraphe	Texte plus long
fake()->name()	Génère un nom	"Ahmed Benali"
fake()->email()	Génère un email	"test@mail.com”
fake()->word()	Génère un mot	"database"
fake()->text()	Génère un texte	texte aléatoire

6. Base de Données & Eloquent ORM

Un Model représente une table dans la base de données.

php artisan make:model User



php artisan make:model User -[c | mc | mfsc | cr]

Variable	Rôle
$table	Nom de la table associée
$primaryKey	Clé primaire de la table
$fillable	Champs autorisés pour le mass assignment
$guarded	Champs protégés du mass assignment
$hidden	Champs cachés dans les JSON
$casts	Conversion automatique des types
$timestamps	Gestion automatique created_at et updated_at

One To One

Un utilisateur possède un seul profil.

Dans User:

```
public function profile()
{
    return $this->hasOne(Profile::class);
}
```

Dans le fichier de migration:

$table->foreignId('user_id')->constrained();

One To Many

Un utilisateur peut avoir plusieurs posts.

Dans User:

```
public function posts()
{
    return $this->hasMany(Post::class);
}
```

Dans Post:

```
public function user()
{
    return $this->belongsTo(User::class);
}
```

Many To Many

Un post peut avoir plusieurs tags.

Table pivot : **post_tag**

Dans le fichier de migration:

```
Schema::create('post_tag', function (Blueprint $table) {
    $table->id();

    $table->foreignId('post_id')->constrained();
    $table->foreignId('tag_id')->constrained();

    $table->timestamps();
});
```

Dans le model Post:

```
public function tags()
{
    return $this->belongsToMany(Tag::class);
}
```

Dans le model Tag:

```
public function posts()
{
    return $this->belongsToMany(Post::class);
}
```

Règle Laravel importante

Pour les relations Many-to-Many, le nom de la table pivot doit respecter l'ordre alphabétique des noms des tables.

Donc : *post_tag* , et pas *tag_post*

Mini TP : Models, Migrations, Relations, Factories et Seeders

Objectif :

- Configurer la connexion à une base de données dans Laravel
- Créer des Models et Migrations avec Artisan
- Mettre en place une relation entre les modèles
- Générer des données fictives avec Factories
- Remplir la base de données avec des Seeders
- Mettre en place le schéma de base de données dans un SGBD

Etapes :

Implémentez les modèles de l'application de gestion de blog en respectant les relations suivantes :

- Un Post peut avoir plusieurs Comments.
- Un Comment appartient à un seul Post.